

Handwritten text at the top right, possibly a date or header.

Handwritten text in the upper left section.

Handwritten text in the upper middle section.

Handwritten text in the middle left section.

Handwritten text in the middle left section.

Handwritten text in the middle left section.

Handwritten text in the middle left section.

Handwritten text in the middle right section, possibly a list or description.

Handwritten text in the middle left section.

Handwritten text in the lower middle section, possibly a signature or large note.

Handwritten text in the lower left section.

Handwritten text in the lower left section.

Handwritten text in the lower right section.

Handwritten text in the lower right section.

(1A)

add recursion from the grammar

$$A \rightarrow A\alpha / \beta$$

$$A \rightarrow A\alpha$$

$$A \rightarrow \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A$$

First (S) : S  
first (B) :  
first (C) :  
first (C) :  
first (C) :  
first (C) :

(5A)

Given

$$S \rightarrow iete / ietsee / a$$

By applying sets

$$S \rightarrow \alpha / \beta \epsilon / a$$

$$S \rightarrow \alpha$$

$$S \rightarrow \beta \cdot a$$

$$S \rightarrow a$$

(6A)

Given that

$$S \rightarrow aBDh$$

$$B \rightarrow cC$$

$$C \rightarrow bc / e$$

$$D \rightarrow EF$$

$$E \rightarrow g / e$$

$$F \rightarrow f / e$$



|             |          |              |              |
|-------------|----------|--------------|--------------|
| first (A) : | { a }    | follow (A) : | { \$ }       |
| first (B) : | { c }    | follow (B) : | { g, +, \$ } |
| first (C) : | { b, e } | follow (C) : | { g, +, \$ } |
| first (D) : | { g, e } | follow (D) : | { g, +, \$ } |
| first (E) : | { g, e } | follow (E) : | { +, g, \$ } |
| first (F) : | { +, e } | follow (F) : | { +, g, \$ } |

Given that

$$E \rightarrow E a / E b / c d$$

left recursion :

$$E \rightarrow E a / E b / c d$$

$$E \rightarrow E a$$

$$E \rightarrow E b$$

$$E \rightarrow c d$$

$$E \rightarrow b e$$

$$E \rightarrow a e$$

$$E \rightarrow a e / b e / e$$

It contains immediate left recursion, so  $E \rightarrow E a / E b / c d$ .

Given that

$$E \rightarrow T E', E' \rightarrow + T E' / e$$

find first (E')

600

100

2. string (a) is Va

2. string (a) is valid

01024-2

formed on shift

formed on shift

Grav

Gravel

in the stack

in the stack  
on do the chif  
n 2112 to RHC.

on do the shift  
in LHS to RHS.  
RHS element r

greater than the

greater than the  
d.

tion  
aid

tion  
aid



Given that:

$$S' \rightarrow S$$
$$S \rightarrow cC$$
$$C \rightarrow cC/d$$
$$\text{item} = [S' \rightarrow \cdot S]$$

find closure( $[S' \rightarrow \cdot S]$ ) list all the LR(0)

closure:

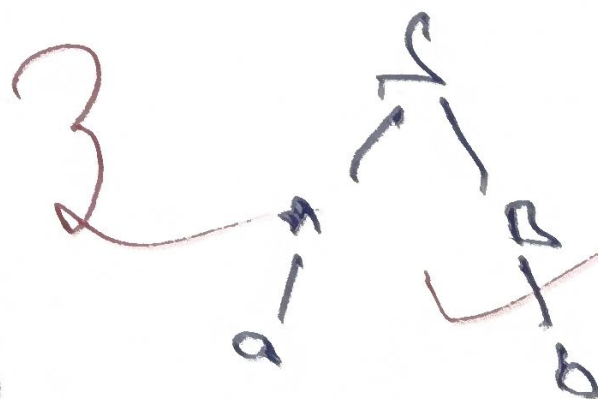
$$S' \rightarrow S$$
$$S \rightarrow \cdot cC$$
$$C \rightarrow \cdot cC/d$$

first set for the non-terminal in  
 $S \rightarrow AB, A \rightarrow a, B \rightarrow b$ .

grammar

$$S \rightarrow asb/ab$$

check whether  $aabb$  is accepted.



$aabb$  is not possible

first( $S$ ):  $\{a\}$

first( $A$ ):  $\{a\}$

first( $B$ ):  $\{b\}$

(13A) Given that

$S \rightarrow A$   
 $A \rightarrow aB / a d$   
 $B \rightarrow b$   
 $C \rightarrow g$

$A \rightarrow A d$   
 $A \rightarrow d A$

Now,

$first(S) : \{a\}$   
 $first(A) : \{a\}$   
 $first(A') : \{d\}$   
 $first(B) : \{b\}$   
 $first(C) : \{g\}$

$follow(C) : \{\$ \}$   
 $follow(A) : \{b, \$ \}$   
 $follow(A') : \{b, \$ \}$   
 $follow(B) : \{b, \$ \}$   
 $follow(C) : \{g, b, \$ \}$

(14A)

Simple lex code to parse Expression with addition.

Code  $\div$

$LDF : R_1, R_2$

$ADF : A, R_1, R_2$

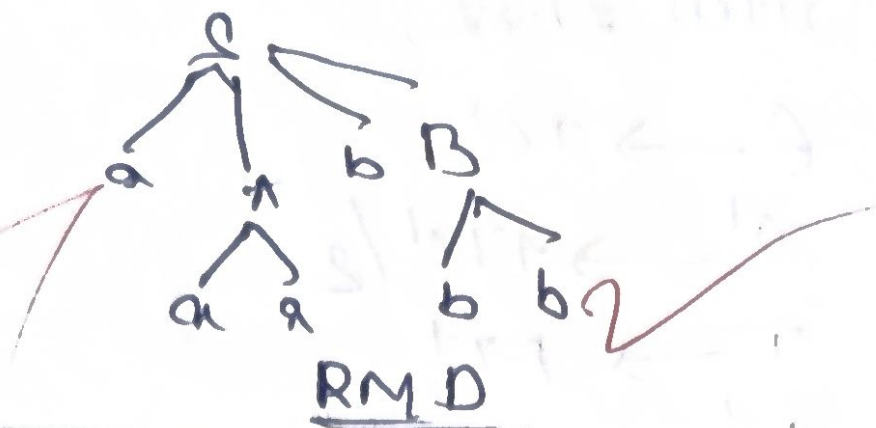
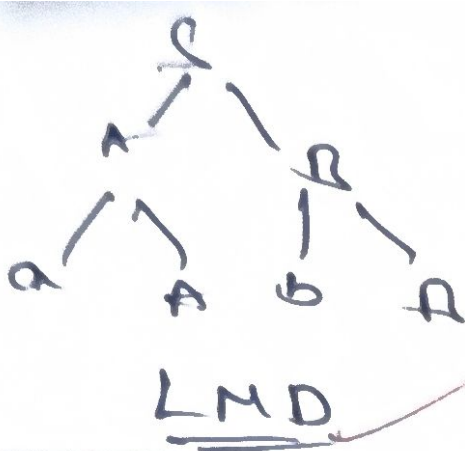
$STF : id, R_1$

(15A)

Given that,

$S \rightarrow AB$   
 $A \rightarrow aA / a$   
 $B \rightarrow bB / b$





(6A)

$S \rightarrow AB$

$A \rightarrow aA \mid \epsilon$

$B \rightarrow bB \mid \epsilon$

$\text{first}(S) = \{a\}$

$\text{follow}(\epsilon) = \{\$ \}$

$\text{first}(A) = \{a, \epsilon\}$

$\text{follow}(A) = \{ \}$

$\text{first}(B) = \{b, \epsilon\}$

$\text{follow}(B) = \{ \}$

(7A)

Rules of Left Recursion and Left Factor  
ing:

Left Recursion:

Given that,

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (e) / id$$

- (i) Recursion - Descent parsing.
- (ii) Reduce (Bottom-up) parsing.

|              |   |                     |                 |                   |
|--------------|---|---------------------|-----------------|-------------------|
| First $(E)$  | : | $\{ (, id \}$       | Follow $(E)$ :  | $\{ \$, \}$       |
| first $(E')$ | : | $\{ +, \epsilon \}$ | Follow $(E')$ : | $\{ \$, \}$       |
| first $(T)$  | : | $\{ (, id \}$       | Follow $(T)$ :  | $\{ \$, \}$       |
| first $(T')$ | : | $\{ *, \epsilon \}$ | Follow $(T')$ : | $\{ +, \$, \}$    |
| first $(F)$  | : | $\{ (, id \}$       | Follow $(F)$ :  | $\{ *, +, \$, \}$ |

Terminals

| Nonterminal | . | id                       | +                     | *                     | (                        | ) | \$ |
|-------------|---|--------------------------|-----------------------|-----------------------|--------------------------|---|----|
| E           |   | $E \rightarrow TE'$      |                       |                       | $E \rightarrow TE'$      |   |    |
| E'          |   |                          | $E' \rightarrow +TE'$ |                       |                          |   |    |
| T           |   | $T \rightarrow FT'$      |                       |                       | $T \rightarrow FT'$      |   |    |
| T'          |   |                          |                       | $T' \rightarrow *FT'$ |                          |   |    |
| F           |   | $F \rightarrow (e) / id$ |                       |                       | $F \rightarrow (e) / id$ |   |    |





e + id<sub>2</sub>

e + e

e

e +

e + id<sub>3</sub>

e + e

e

\* id<sub>3</sub> \$

+ id<sub>3</sub> \$

\* id<sub>3</sub> \$

id<sub>3</sub> \$

\$

\$

\$

Shift

Reduce

id<sub>2</sub> → e

Reduce

e → e + e

Shift

Shift

Reduce

id<sub>3</sub> → e

Accepted

201

```
#include <stdio.h>
```

```
int main () {
```

```
    int a=10, b=20;
```

```
    int sum = a+b;
```

```
    printf ("%d", sum);
```

```
    return 0;
```

```
}
```

(a)

Lexical Analysis



Syntax Analysis



Semantic Analyzer



Intermediate Code Generator



Code Optimization



Code generator



1. 1000 1000 1000  
 2. 1000 1000 1000  
 3. 1000 1000 1000



1. 1000 1000 1000  
 2. 1000 1000 1000  
 3. 1000 1000 1000

① Given.

Com-ack  
 Seq-ack

→ seq-ack  
 ↓

G → seq-ack  
 E → seq-ack

T → seq-ack

9